DM_Assignment 1: Build the Model from Scratch

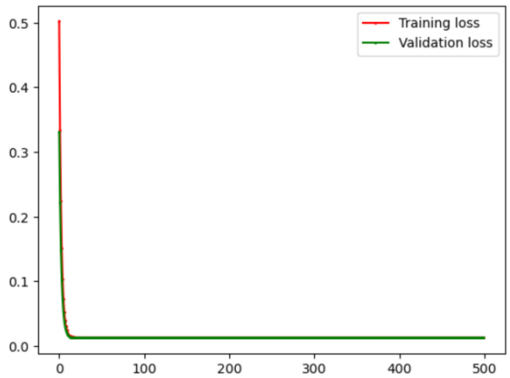
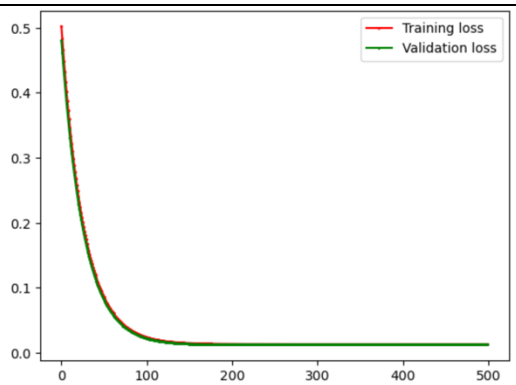
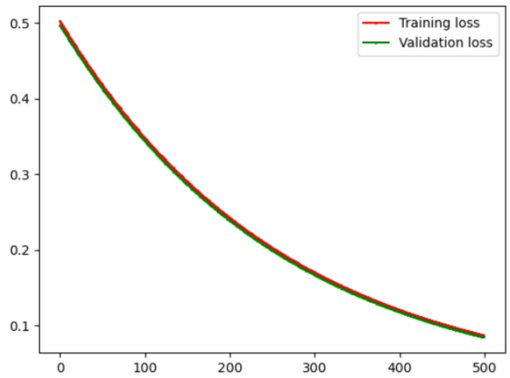
414551017 陳冠甫

GitHub：https://github.com/Patrickchen2000/DM2026-Assignment-1

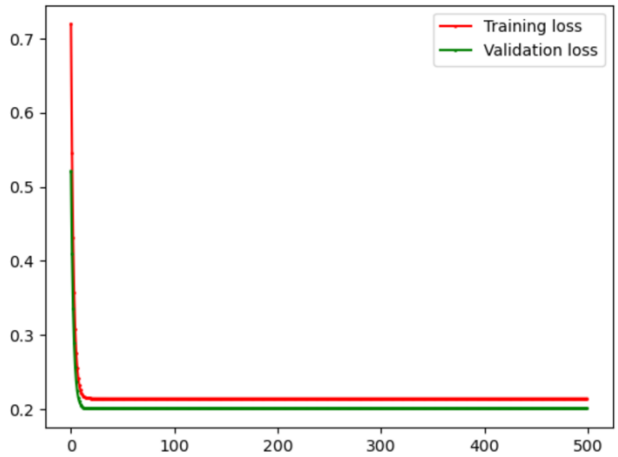
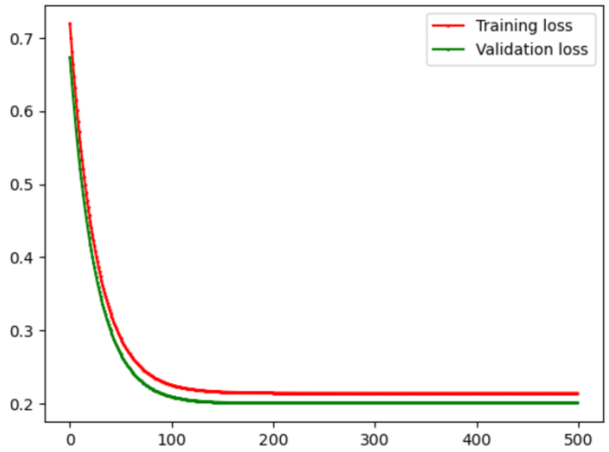
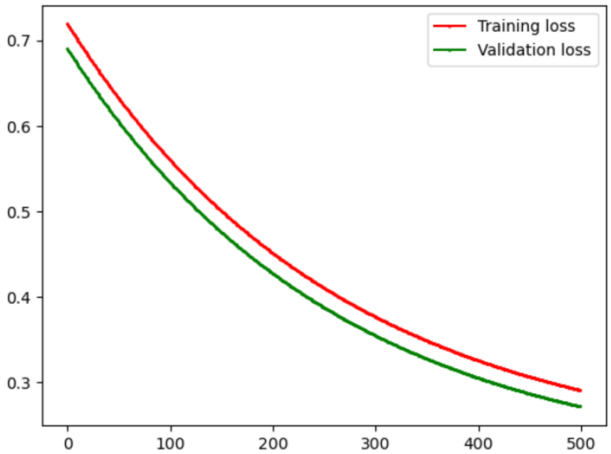
1. Linear Regression (20 points):

a). Plot the loss curve for each learning rate

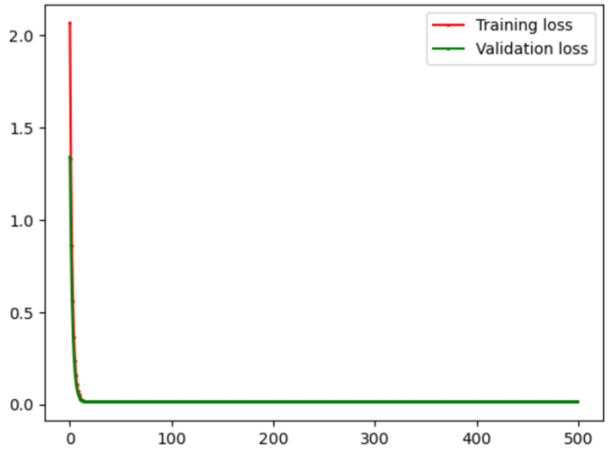
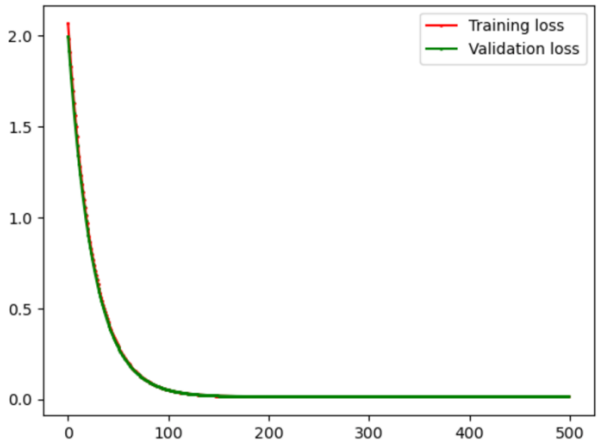
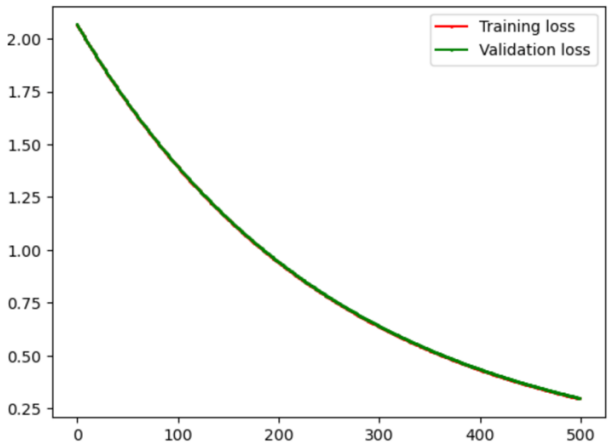
Dataset : linear_data_A

learning rate	loss curves
0.1	
0.01	
0.001	

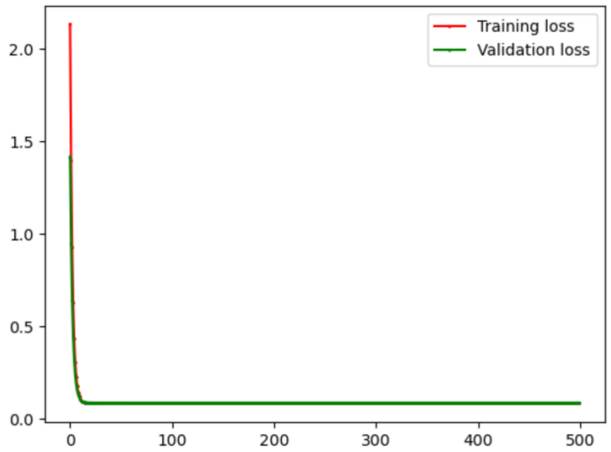
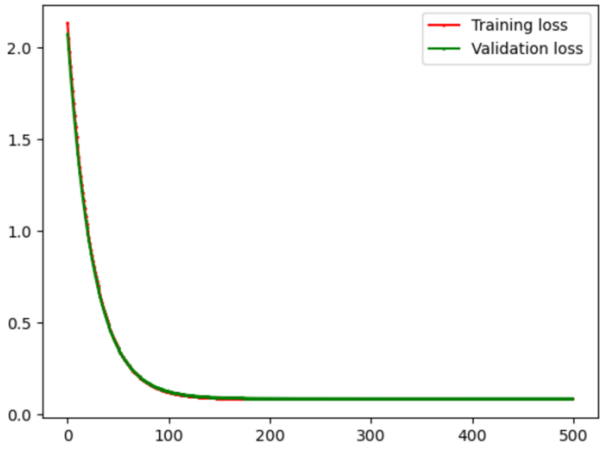
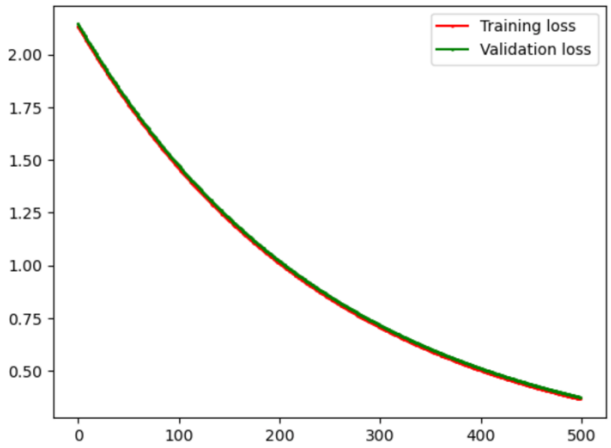
Dataset : linear_data_B

learning rate	loss curves
0.1	
0.01	
0.001	

Dataset : linear_data_C

learning rate	loss curves
0.1	
0.01	
0.001	

Dataset : linear_data_D

learning rate	loss curves
0.1	
0.01	
0.001	

b). Show the evaluations in different learning rate

Dataset : linear_data_A

learning rate	evaluations

0.1	`=== Data A - Linear Regression Evaluation (lr=0.1) ===`
	`MSE       : 0.0134`
	`MAE       : 0.1010`
	`RMSE      : 0.1156`
	`R-squared : 0.5690`

0.01	`=== Data A - Linear Regression Evaluation (lr=0.01) ===`
	`MSE       : 0.0134`
	`MAE       : 0.1010`
	`RMSE      : 0.1156`
	`R-squared : 0.5690`

0.001	`=== Data A - Linear Regression Evaluation (lr=0.001) ===`
	`MSE       : 0.0832`
	`MAE       : 0.2369`
	`RMSE      : 0.2884`
	`R-squared : -1.6836`

Dataset : linear_data_B

learning rate	evaluations

0.1	`=== Data B - Linear Regression Evaluation (lr=0.1) ===`
	`MSE       : 0.2137`
	`MAE       : 0.4038`
	`RMSE      : 0.4623`
	`R-squared : 0.0448`

0.01	`=== Data B - Linear Regression Evaluation (lr=0.01) ===`
	`MSE       : 0.2137`
	`MAE       : 0.4038`
	`RMSE      : 0.4623`
	`R-squared : 0.0448`

0.001	`=== Data B - Linear Regression Evaluation (lr=0.001) ===`
	`MSE       : 0.2687`
	`MAE       : 0.4404`
	`RMSE      : 0.5184`
	`R-squared : -0.2011`

Dataset : linear_data_C

learning rate	evaluations

0.1	`=== Data C - Linear Regression Evaluation (lr=0.1) ===`
	`MSE       : 0.0132`
	`MAE       : 0.0987`
	`RMSE      : 0.1148`
	`R-squared : 0.9954`

0.01	`=== Data C - Linear Regression Evaluation (lr=0.01) ===`
	`MSE       : 0.0132`
	`MAE       : 0.0987`
	`RMSE      : 0.1148`
	`R-squared : 0.9954`

0.001	`=== Data C - Linear Regression Evaluation (lr=0.001) ===`
	`MSE       : 0.2852`
	`MAE       : 0.4273`
	`RMSE      : 0.5341`
	`R-squared : 0.8994`

Dataset : linear_data_D

learning rate	evaluations

0.1	`=== Data D - Linear Regression Evaluation (lr=0.1) ===`
	`MSE       : 0.0824`
	`MAE       : 0.2467`
	`RMSE      : 0.2870`
	`R-squared : 0.9716`

0.01	`=== Data D - Linear Regression Evaluation (lr=0.01) ===`
	`MSE       : 0.0824`
	`MAE       : 0.2467`
	`RMSE      : 0.2870`
	`R-squared : 0.9716`

0.001	`=== Data D - Linear Regression Evaluation (lr=0.001) ===`
	`MSE       : 0.3540`
	`MAE       : 0.4778`
	`RMSE      : 0.5950`
	`R-squared : 0.8780`

c). Describe your observations in 1(a) and 1(b).

Across all four datasets, there is a clear and consistent relationship between the loss curves in (a) and the evaluation metrics in (b). Learning rates of 0.1 and 0.01 both lead to fast and stable convergence, resulting in significantly better regression performance compared to 0.001. Although both achieve similar final results, a learning rate of 0.1 is more efficient, as it reaches convergence in fewer iterations.

Among the datasets, linear_data_C demonstrates the best performance, achieving the lowest errors (MSE = 0.0132, MAE = 0.0987, RMSE = 0.1148) and the highest R-squared (0.9954). This is well supported by its loss curves, where both training and validation losses rapidly decrease and approach near zero, indicating an excellent linear fit.

linear_data_D also shows strong performance with a high R-squared (0.9716), but its error values are consistently higher than those of linear_data_C, suggesting slightly more noise or variability in the data. In contrast, linear_data_A achieves only moderate performance ($R^2 \approx 0.57$), indicating that the linear model captures the trend partially but not perfectly. linear_data_B performs the worst, with high error values and a very low R-squared (0.0448), implying that the data may have weak linearity or higher noise, making it difficult for a linear model to fit well.

For all datasets, a learning rate of 0.001 results in much slower loss reduction and inferior evaluation metrics. This indicates that the model has

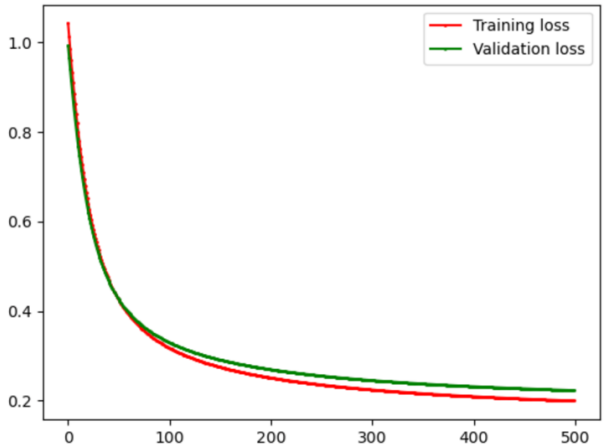
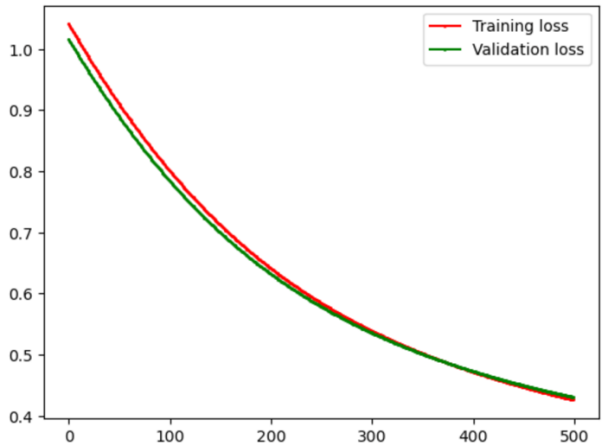
not fully converged within the fixed number of training iterations, leading to underfitting.

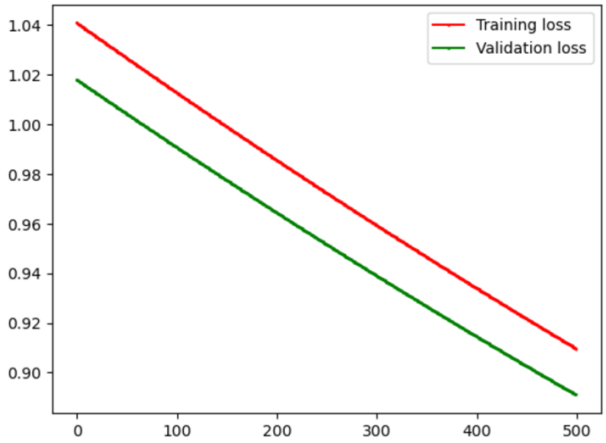
In conclusion, both learning rate and dataset characteristics significantly affect performance. A learning rate of 0.1 provides the best trade-off between convergence speed and accuracy, while linear_data_C is the most suitable dataset for linear regression due to its strong linear relationship. Therefore, the combination of linear_data_C with learning rate 0.1 yields the best overall result.

2. Logistic Regression (20 points):

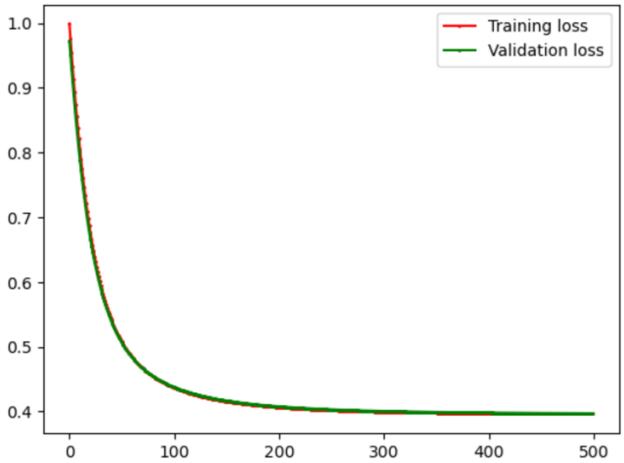
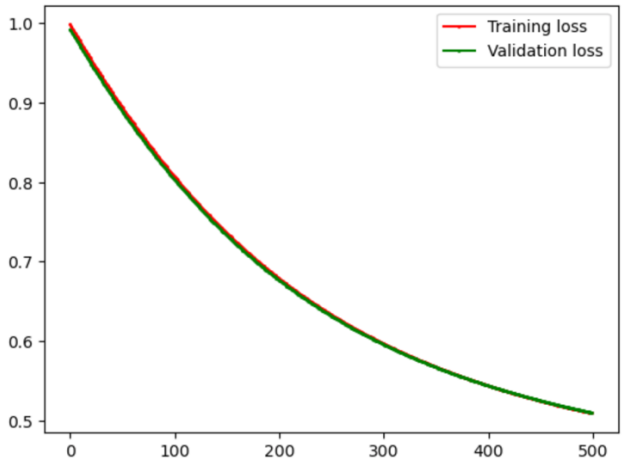
a). Plot the loss curve for each learning rate

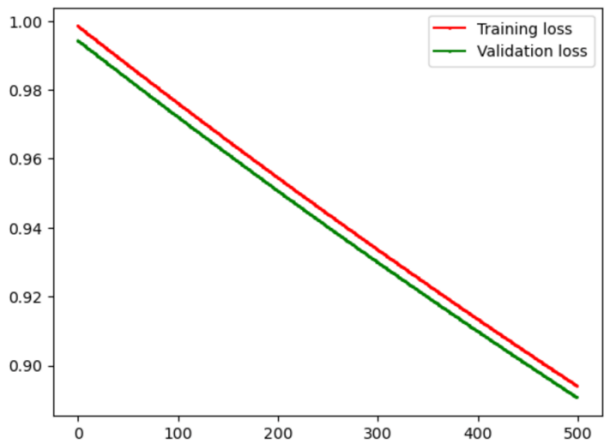
Dataset : logistic_data_A

learning rate	loss curves
0.1	
0.01	

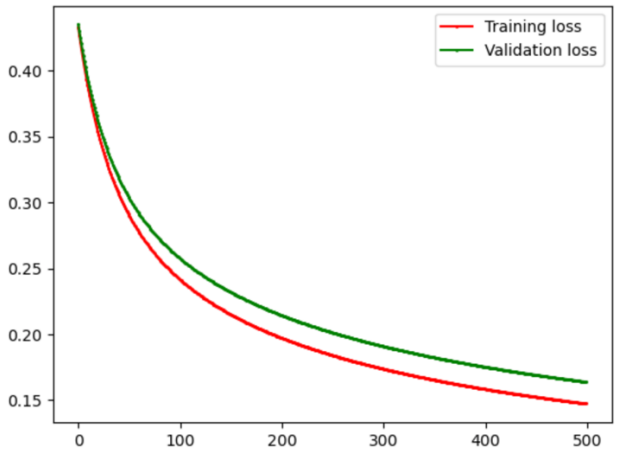
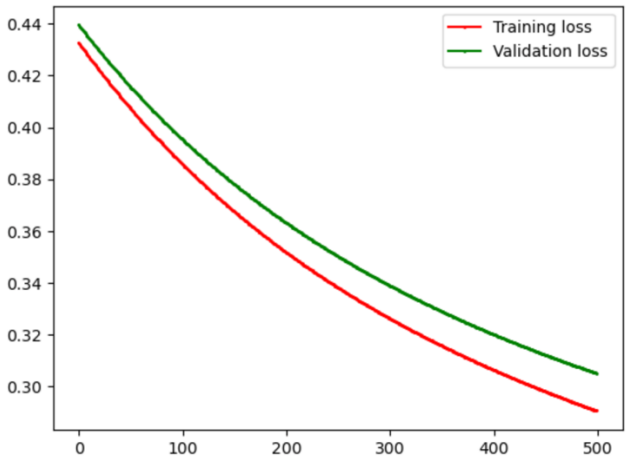
0.001	

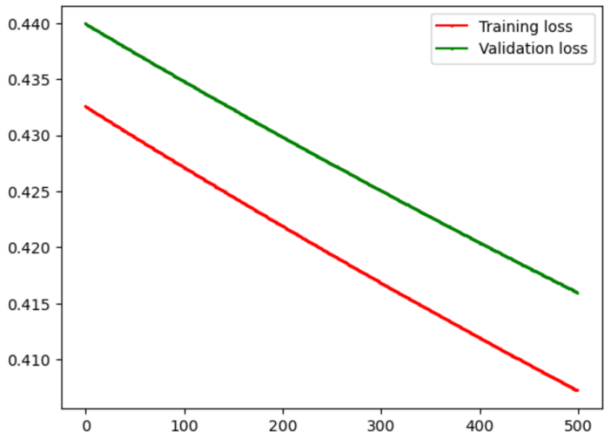
Dataset : logistic_data_B

learning rate	**loss curves**
0.1	
0.01	

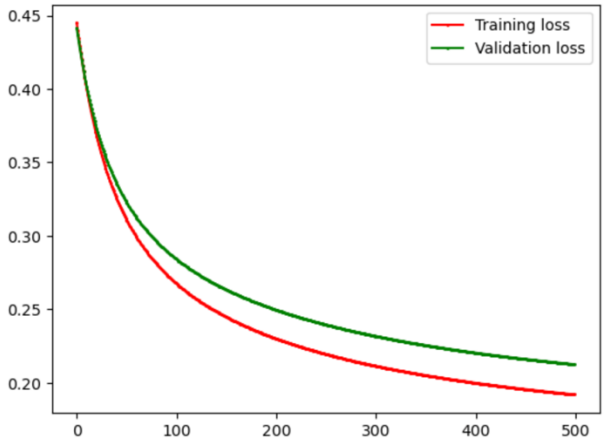
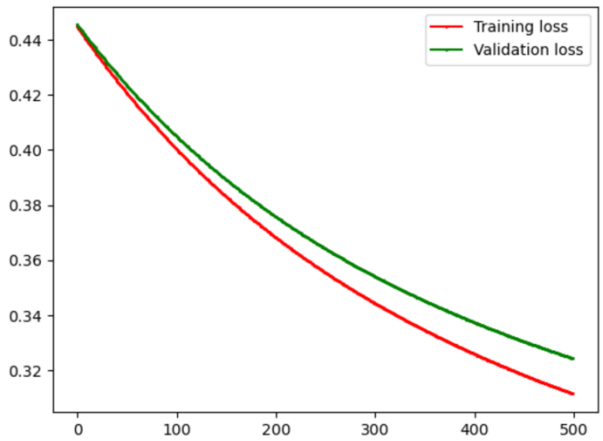
0.001	

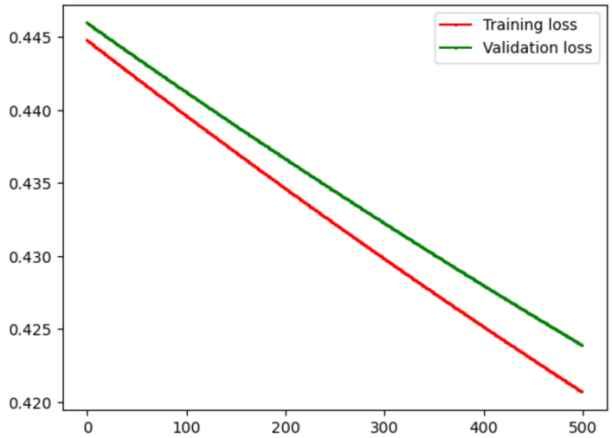
Dataset : logistic_data_C

learning rate	**loss curves**
0.1	
0.01	

0.001	

Dataset : logistic_data_D

learning rate	**loss curves**
0.1	
0.01	

0.001	

b). Show the evaluations in different learning rate

Dataset : logistic_data_A

learning rate	evaluations
0.1	Logistic Regression (lr=0.1) Accuracy : 0.9050 Precision : 0.9087 Recall : 0.9171 F1-score : 0.9128
0.01	Logistic Regression Evaluation Accuracy : 0.8775 Precision : 0.8784 Recall : 0.8986 F1-score : 0.8884
0.001	Logistic Regression (lr=0.001) Accuracy : 0.3875 Precision : 0.4364 Recall : 0.4424 F1-score : 0.4394

Dataset : logistic_data_B

learning rate	evaluations
0.1	Logistic Regression Evaluation (Data B, lr=0.1) Accuracy : 0.7800 Precision : 0.7793 Recall : 0.8160 F1-score : 0.7972
0.01	Logistic Regression Evaluation (Data B, lr=0.01) Accuracy : 0.7625 Precision : 0.7600 Recall : 0.8066 F1-score : 0.7826
0.001	Logistic Regression Evaluation (Data B, lr=0.001) Accuracy : 0.3925 Precision : 0.4286 Recall : 0.4387 F1-score : 0.4336

Dataset : logistic_data_C

learning rate	evaluations
0.1	Logistic Regression Evaluation (Data C, lr=0.1) Accuracy : 0.9762 Precision : 0.9852 Recall : 0.9719 F1-score : 0.9785

0.01	Logistic Regression Evaluation (Data C, lr=0.01) Accuracy : 0.9456 Precision : 0.9707 Recall : 0.9303 F1-score : 0.9500
0.001	Logistic Regression Evaluation (Data C, lr=0.001) Accuracy : 0.8406 Precision : 0.8774 Recall : 0.8290 F1-score : 0.8525

Dataset : logistic_data_D

learning rate	**evaluations**
0.1	Logistic Regression Evaluation (Data D, lr=0.1) Accuracy : 0.9225 Precision : 0.9330 Recall : 0.9266 F1-score : 0.9298
0.01	Logistic Regression Evaluation (Data D, lr=0.01) Accuracy : 0.9113 Precision : 0.9346 Recall : 0.9029 F1-score : 0.9185
0.001	Logistic Regression Evaluation (Data D, lr=0.001) Accuracy : 0.8319 Precision : 0.8668 Recall : 0.8228 F1-score : 0.8442

c). Describe your observations in 2(a) and 2(b).

From the experimental results, the learning rate of 0.1 provides the best overall trade-off between convergence efficiency and model performance. Specifically, it consistently achieves the fastest reduction in loss and yields the highest or near-highest evaluation metrics across all datasets. In contrast, a learning rate of 0.01 demonstrates stable but slower convergence, while 0.001 results in significantly slower loss reduction and inferior final performance due to insufficient convergence within the same training budget.

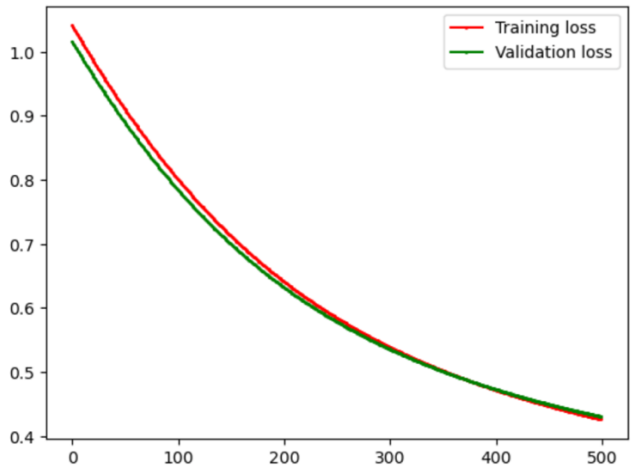
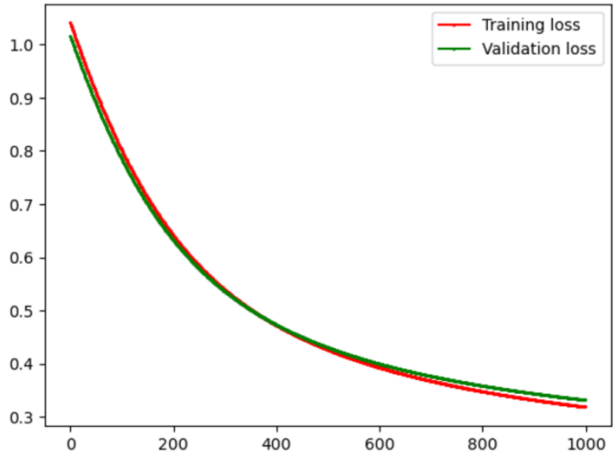
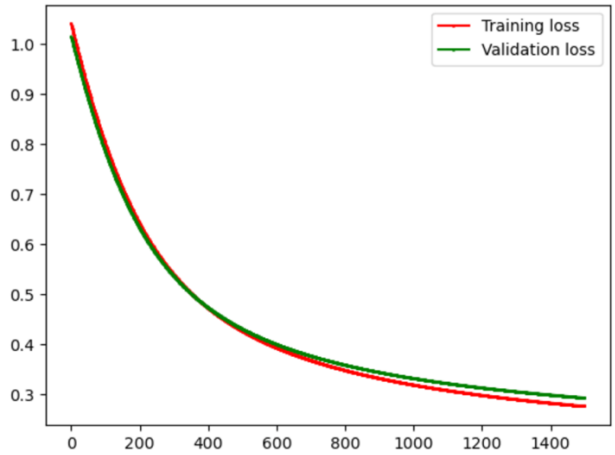
Across the four datasets, logistic_data_C exhibits the best overall performance. At a learning rate of 0.1, it achieves the highest classification metrics (Accuracy = 0.9762, Precision = 0.9852, Recall = 0.9719, F1-score = 0.9785), which is consistent with its well-behaved loss curves and low final loss values. logistic_data_D shows the second-best performance, also demonstrating stable convergence and strong evaluation results. logistic_data_A performs reasonably well but remains slightly inferior to datasets C and D. In contrast, logistic_data_B yields the weakest performance across all learning rates, indicating that this dataset is more challenging for the model to separate effectively.

Regarding generalization behavior, a mild tendency toward overfitting can be observed in logistic_data_C and logistic_data_D when using a learning rate of 0.1. In these cases, the training loss remains consistently lower than the validation loss, with a noticeable gap throughout the training process. However, since the final evaluation metrics remain high, this phenomenon is better characterized as mild overfitting rather than severe overfitting. On the other hand, logistic_data_B exhibits minimal discrepancy between training and validation loss but suffers from significantly lower performance, suggesting that its primary limitation is underfitting or poor data separability rather than overfitting.

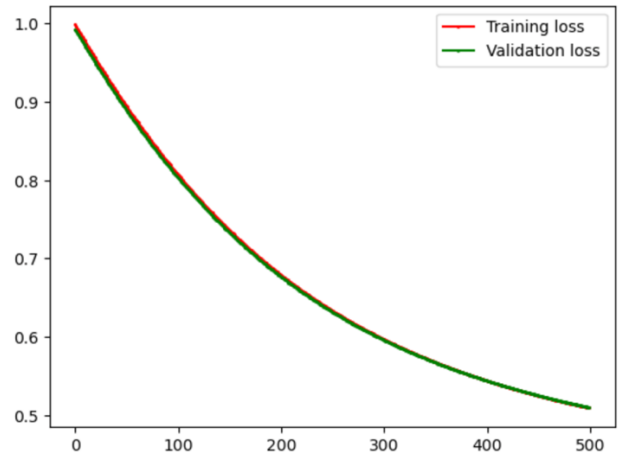
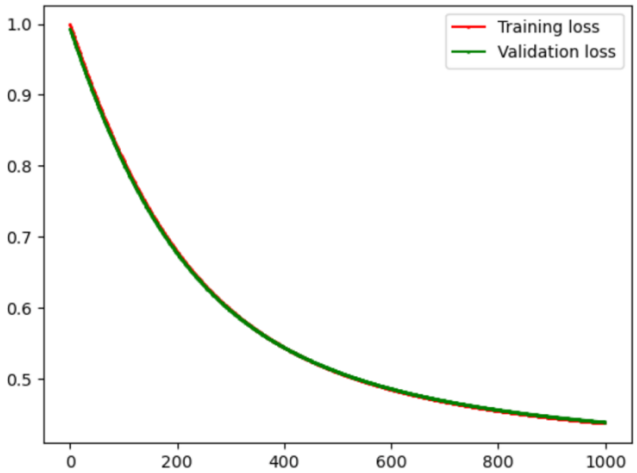
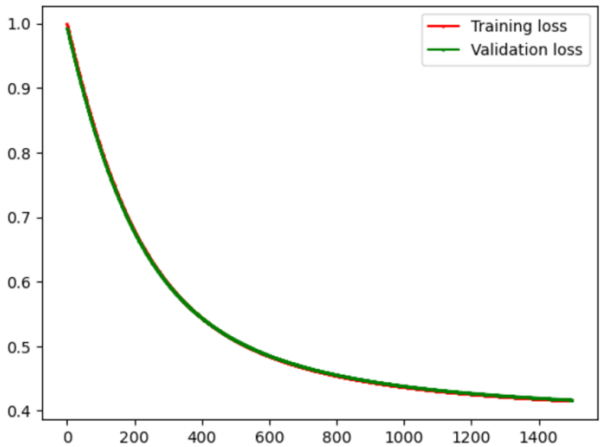
In summary, the most effective and efficient configuration is achieved with logistic_data_C at a learning rate of 0.1, while the most noticeable overfitting tendency appears in datasets C and D under higher learning rates.

d). Plot the loss curve for each setting(using a fixed learning rate of 0.01)

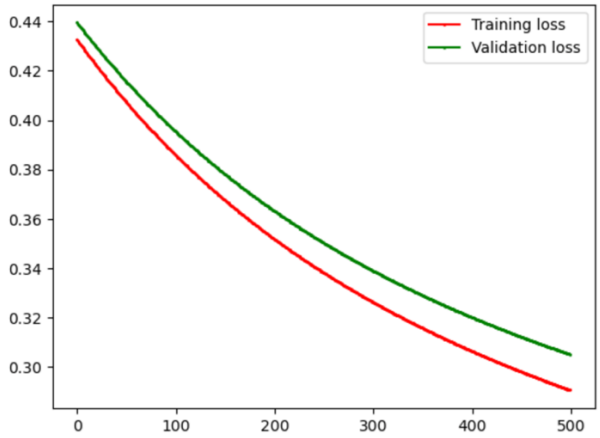
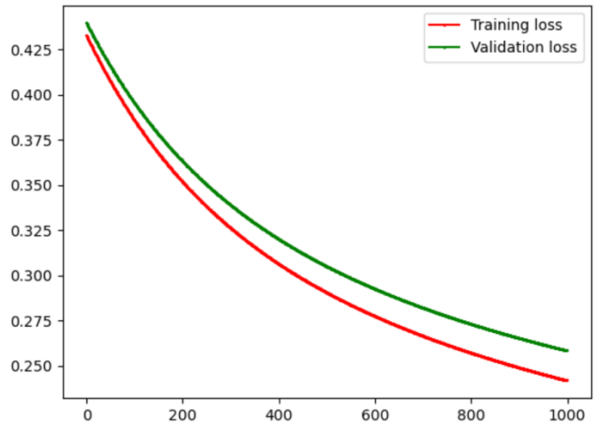
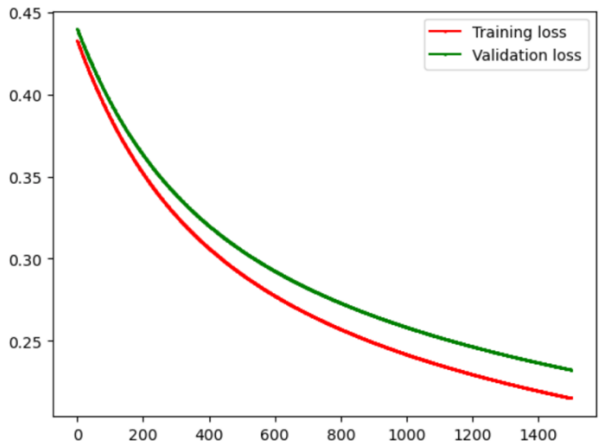
Dataset : logistic_data_A

iterations	loss curves
500	
1000	
1500	

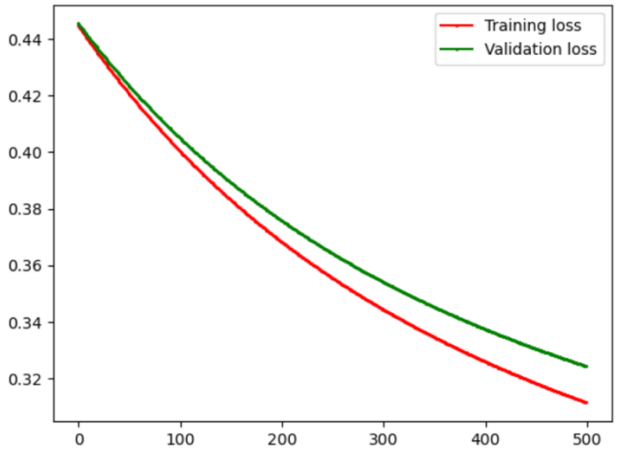
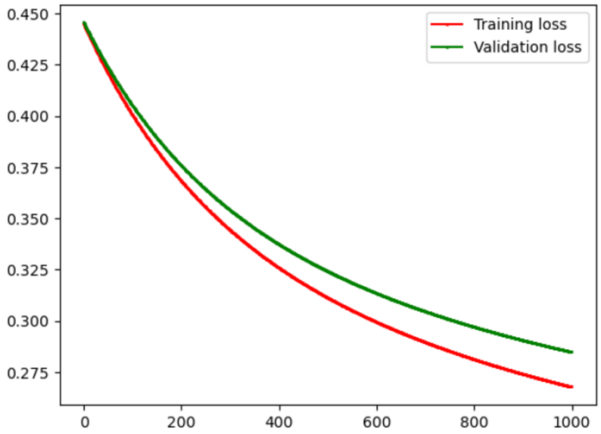
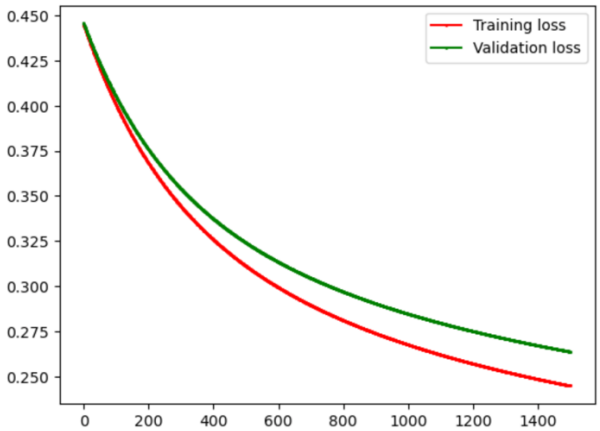
Dataset : logistic_data_B

iterations	loss curves
500	
1000	
1500	

Dataset : logistic_data_C

iterations	loss curves
500	
1000	
1500	

Dataset : logistic_data_D

iterations	loss curves
500	
1000	
1500	

e). Following 2(d), show the evaluations

Dataset : logistic_data_A

iterations	evaluations
500	Logistic Regression Evaluation (Data A, n_iteration=500) Accuracy : 0.8775 Precision : 0.8784 Recall : 0.8986 F1-score : 0.8884
1000	Logistic Regression Evaluation (Data A, n_iteration=1000) Accuracy : 0.9025 Precision : 0.9045 Recall : 0.9171 F1-score : 0.9108
1500	Logistic Regression Evaluation (Data A, n_iteration=1500) Accuracy : 0.9025 Precision : 0.9083 Recall : 0.9124 F1-score : 0.9103

Dataset : logistic_data_B

iterations	evaluations
500	Logistic Regression Evaluation (Data B, n_iteration=500) Accuracy : 0.7625 Precision : 0.7600 Recall : 0.8066 F1-score : 0.7826
1000	Logistic Regression Evaluation (Data B, n_iteration=1000) Accuracy : 0.7775 Precision : 0.7783 Recall : 0.8113 F1-score : 0.7945

1500	Logistic Regression Evaluation (Data B, n_iteration=1500) Accuracy : 0.7775 Precision : 0.7783 Recall : 0.8113 F1-score : 0.7945

Dataset : logistic_data_C

iterations	evaluations
500	Logistic Regression Evaluation (Data C, n_iteration=500) Accuracy : 0.9456 Precision : 0.9707 Recall : 0.9303 F1-score : 0.9500
1000	Logistic Regression Evaluation (Data C, n_iteration=1000) Accuracy : 0.9744 Precision : 0.9885 Recall : 0.9651 F1-score : 0.9767
1500	Logistic Regression Evaluation (Data C, n_iteration=1500) Accuracy : 0.9788 Precision : 0.9897 Recall : 0.9719 F1-score : 0.9807

Dataset : logistic_data_D

iterations	evaluations
500	Logistic Regression Evaluation (Data D, n_iteration=500) Accuracy : 0.9113 Precision : 0.9346 Recall : 0.9029 F1-score : 0.9185

1000	Logistic Regression Evaluation (Data D, n_iteration=1000) Accuracy : 0.9263 Precision : 0.9394 Recall : 0.9266 F1-score : 0.9330
1500	Logistic Regression Evaluation (Data D, n_iteration=1500) Accuracy : 0.9256 Precision : 0.9393 Recall : 0.9255 F1-score : 0.9323

f). Describe your observations in 2(d) and 2(e).

The results show that the influence of increasing the number of iterations differs across datasets. For “logistic_data_A” and “logistic_data_B”, the loss curves converge relatively early, and the evaluation results show only limited improvement as the number of iterations increases from 500 to 1500. This suggests that these datasets reach satisfactory performance within fewer training iterations, and additional optimization brings only marginal benefit.

In contrast, “logistic_data_C” and “logistic_data_D” benefit more clearly from extended training. Their loss curves continue to decrease as the number of iterations increases, indicating that the models have not fully converged at earlier stages. This trend is consistent with the evaluation results, which improve progressively with longer training. In particular, “logistic_data_C” achieves the best overall performance at 1500 iterations, with an Accuracy of 0.9788, Precision of 0.9897, Recall of 0.9719, and an F1-score of 0.9807. This demonstrates that increasing the number of training iterations can substantially improve performance for datasets that require a longer optimization process.

Overall, under a fixed learning rate of 0.01, increasing the number of training iterations generally leads to more complete convergence and better classification performance, although the degree of improvement remains dataset-dependent. Datasets with early convergence gain little from additional iterations, whereas datasets with more gradual optimization behavior benefit noticeably from longer training.

3. Data Preprocessing (20 points):

a). Report the **median** and **standard deviation** for columns with missing values.

```
Columns with missing values before imputation:
SepalLengthCm: median = 6.3000, std = 1.0371
SepalWidthCm: median = 2.9000, std = 0.3896
PetalLengthCm: median = 5.0856, std = 1.5828
PetalWidthCm: median = 1.6000, std = 0.7067
BranchLength: median = 16.3000, std = 1.0352
```

b). **Replace the missing values** using "**Nearest Neighbors Imputation**" for columns with missing values. Report the **median** and **standard deviation** of those columns.

```
Columns with missing values after KNN imputation:
SepalLengthCm: median = 6.3000, std = 1.0106
SepalWidthCm: median = 2.9000, std = 0.3737
PetalLengthCm: median = 5.0595, std = 1.5791
PetalWidthCm: median = 1.7000, std = 0.6951
BranchLength: median = 16.3000, std = 1.0126
```

Compare your result with 3(a) and explain your observation.

After applying K-Nearest Neighbors (KNN) imputation, we compare the statistical properties of the dataset with those reported prior to imputation in Section 3(a). The results indicate that the median values across all affected features remain largely unchanged, with only minor variations observed in PetalLengthCm and PetalWidthCm. This suggests that KNN imputation effectively preserves the central tendency of the data distribution.

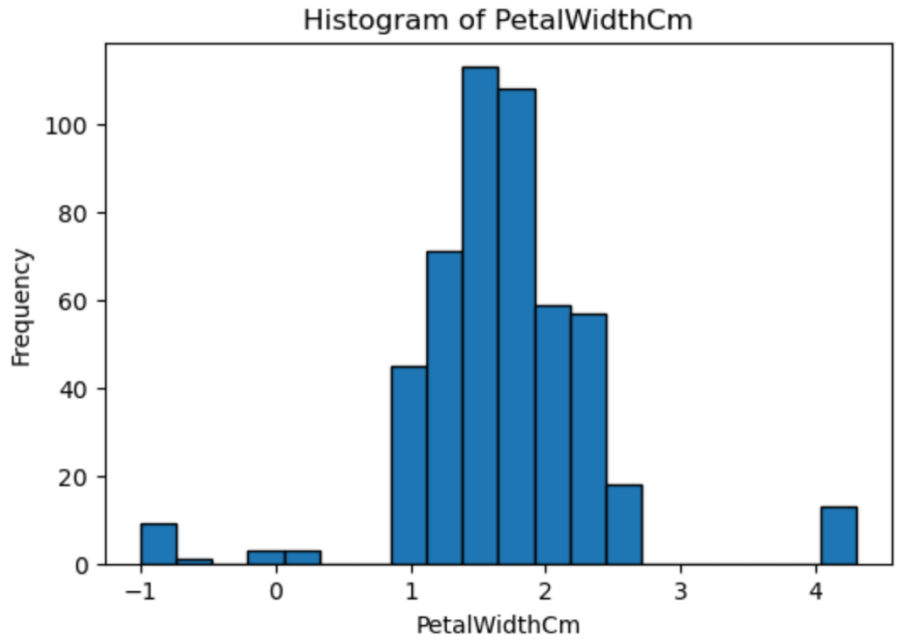
In contrast, a consistent reduction in standard deviation is observed across all features after imputation. This decrease implies that the imputed values tend to be closer to their neighboring samples, resulting in a slight smoothing effect and reduced variability. Such behavior is expected, as KNN imputation replaces missing values based on similar instances, thereby avoiding extreme or outlier values.

Overall, the results demonstrate that KNN imputation maintains the structural integrity of the dataset while introducing a modest reduction in

variance. This trade-off highlights its effectiveness in generating realistic imputations, albeit at the cost of slightly underrepresenting data dispersion.

4. Data Exploration (20 points):

a). Plot the histogram of the column "PetalWidthCm"



The histogram of PetalWidthCm exhibits a multi-modal distribution, with clear clusters concentrated between 1.0 and 2.5 cm, reflecting the underlying class structure of the Iris dataset. A small number of extreme values are observed outside the typical range, likely resulting from data imputation or noise. Additionally, the distribution shows a slight right skew, though it deviates from normality due to the presence of multiple modes. These findings highlight the heterogeneous nature of the feature and suggest that class-dependent analysis may be more appropriate than assuming a unimodal distribution.

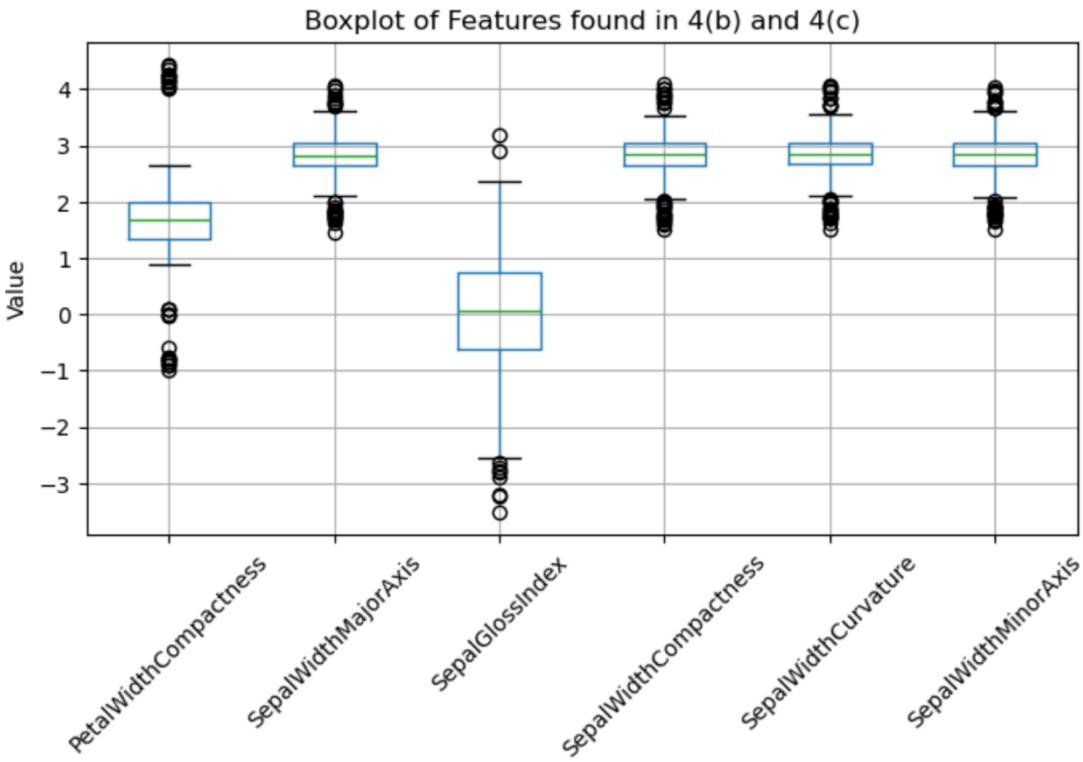
b). List the feature (excluding "PetalWidthCm", "Species", and "Id") that has the **largest positive correlation with the "PetalWidthCm"** column.

```
Feature with the largest positive correlation: PetalWidthCompactness
Correlation value: 0.9909526474865203
```

c). Report the top 5 features with the **strongest negative** correlations with the 'PetalWidthCm' column.

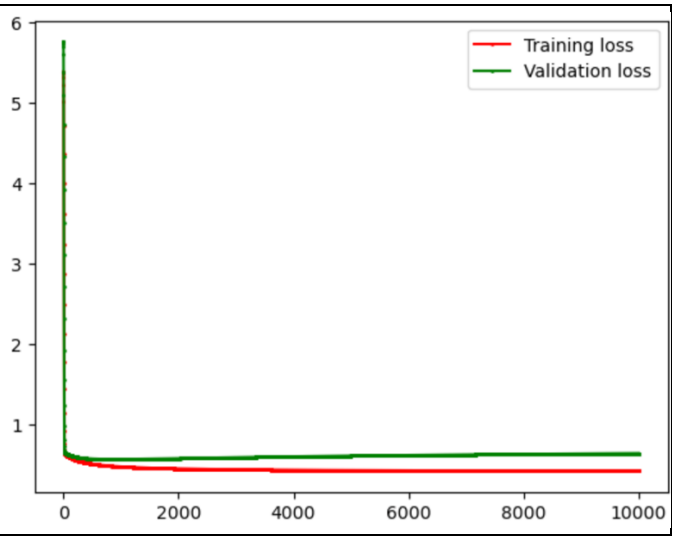
```
Top 5 features with the strongest negative correlations with PetalWidthCm:
SepalWidthMajorAxis       -0.097009
SepalGlossIndex           -0.093236
SepalWidthCompactness     -0.088006
SepalWidthCurvature       -0.081898
SepalWidthMinorAxis       -0.074390
```

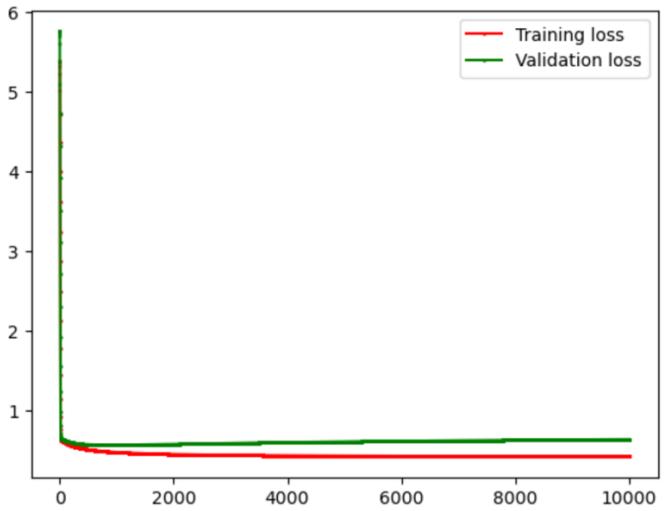
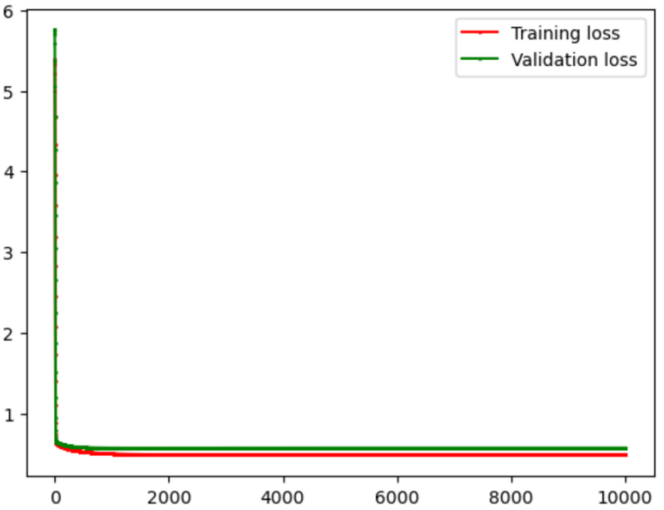
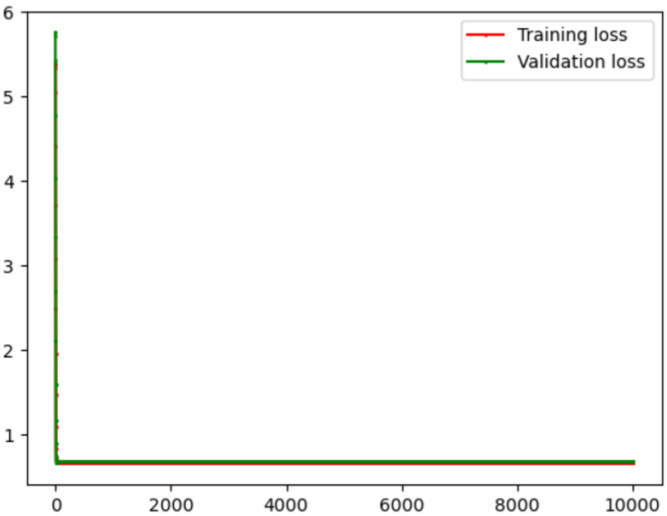
d). Plot the boxplot for all features found in 4(b) and 4(c).



5. Regularization (20 points):

a). L2 Regularization

No Regularization	

L2 (reg_lambda=0.01)	
L2 (reg_lambda=1)	
L2 (reg_lambda=100)	

b). Following 5(a), show the testing results

No Regularization	Model Evaluation Accuracy : 0.7200 Precision : 0.7125 Recall : 0.7500 F1-score : 0.7308
L2 (reg_lambda=0.01)	L2 (lambda=0.01) Model Evaluation Accuracy : 0.7200 Precision : 0.7125 Recall : 0.7500 F1-score : 0.7308
L2 (reg_lambda=1)	L2 (lambda=1) Model Evaluation Accuracy : 0.7400 Precision : 0.7342 Recall : 0.7632 F1-score : 0.7484
L2 (reg_lambda=100)	L2 (lambda=100) Model Evaluation Accuracy : 0.5067 Precision : 0.5067 Recall : 1.0000 F1-score : 0.6726

c). Describe your observations in 5(a) and 5(b).

The results in Fig. 5(a) demonstrate the impact of L2 regularization on training and validation loss. Without regularization, the model converges quickly, but a small gap between training and validation loss can be observed, indicating mild overfitting. When a small regularization strength ($\lambda = 0.01$) is applied, the loss curves remain nearly identical to the unregularized case, suggesting that the regularization effect is negligible at this scale.

With a moderate regularization strength ($\lambda = 1$), both training and validation losses become smoother and closer to each other, indicating improved generalization and reduced overfitting. However, when the regularization strength is excessively large ($\lambda = 100$), the model exhibits

clear underfitting behavior, as both training and validation losses converge to higher values and show limited improvement during training.

The quantitative results in Table 5(b) further support these observations. The model with $\lambda = 1$ achieves the best performance across all metrics (Accuracy = 0.74, F1-score = 0.7484), demonstrating that an appropriate level of regularization improves generalization. In contrast, $\lambda = 0.01$ yields identical performance to no regularization, confirming its limited impact. For $\lambda = 100$, the model performance degrades significantly (Accuracy = 0.5067), with an unusually high recall (1.0) but low precision, indicating that the model tends to over-predict a single class. This behavior is characteristic of an overly constrained model.

Overall, these results highlight the importance of selecting an appropriate regularization strength. While moderate L2 regularization effectively mitigates overfitting and improves generalization, excessive regularization leads to underfitting and degraded performance.